```
bcscale(1); // we need everything to 1 d.p. for the next calls
// Remember 0.1 + 0.7 != 0.8 in floating point arithmetic?
$trickyFloat = bcadd('0.1', '0.7');
var_dump(bccomp($trickyFloat, '0.8')); // now it does!
```

```
Output:
string(40) "1844674407370955161400000000000000000000"
string(38) "83848836698679780063636363636363636363"
string(43) "83848836698679780063636363636363636363.6363"
string(1) "7"
string(78) "850705917302346158473969077842325012490000000000000000000000000
0000000000000000"
int(0)
int(-1)
int(1)
string(6) "200.03"
int(0)
```

Hopefully everything above is self-explanatory. Note that our gotcha from floating-point arithmetic (*0.1 + 0.7* giving a result of *0.7999999999999999*) is handled just fine by `bcadd()` and `bccomp()` on the final two lines. Now you can enjoy stupidly large numbers *and* rational numbers that break IEEE floating point. Remember that BCMath is intended to deal with string representations of numbers; mixing these functions with PHP's core mathematical functions and operators might yield unexpected results. At best the giant numbers output from BCMath will be cast to floats with all the caveats that come thereafter.

```
$largeNum = (string) PHP_INT_MAX;
$largeNum .= '00000000000000000000';
var_dump($largeNum - 1);

$veryLargeNum = $largeNum . str_repeat('0000000000', 30);
var_dump($veryLargeNum + 1);
```

```
Output:
float(9.223372036854776E+38)
float(INF)
```